

GUJARAT TECHNOLOGICAL UNIVERSITY

Navyug Vidyabhavan Trust

C. K. Pithawalla College of Engineering and Technology, Surat

A

Project Report

On

(Student Management and Performance Analysis System)

Prepared as a part of the requirements for the subject of

Python for Data Science (BE05000231)

(PBL - Report_3)

B. E. III, Semester – V

(Computer Engineering)

Submitted by:

Sr. No.	Enrolment No	Name of Student	Division
1	240090107097	Jeel Moradiya	C

Academic year (2026-27)

Table of Contents

Sr. No	Title	Page No
	Abstract	2
1	Introduction	2
2	Problem Statement	2
3	Objectives	3
4	Technologies and Libraries Used	3
5	System Requirements	4
6	System Features	4
7	System WorkFlow	5
8	Dataset	5
9	Data Analysis	7
10	STATISTICAL ANALYSIS	8
11	Data Visualization	10
12	Machine Learning	13
13	NEW STUDENT PREDICTION	14
14	Beyond Syllabus Topic	15
15	Implementation	18
16	Important Code	18
17	Results and Observations	19
18	Advantages	19
19	Limitations	19
20	Future Scope	20
21	Conclusion	20
22	References	21

Abstract

Student Performance Analytics is a Python-based project developed to analyze student academic performance. The system loads student data from a CSV file and performs data cleaning, analysis, statistical calculations, visualization, and machine learning.

The project uses Pandas, NumPy, SciPy, Matplotlib, Seaborn, and Scikit-learn. It calculates total marks, percentage, grade, and Pass/Fail result. Random Forest Classifier is used for Pass/Fail prediction. The project also allows the user to enter a new student's details and marks for prediction.

K-Means Clustering is implemented as the beyond-syllabus topic to group students into Low Performance, Average Performance, and High Performance categories.

1. Introduction

Student academic performance is an important part of the education system. Student marks contain useful information that can be analyzed to understand individual and overall performance.

The Student Performance Analytics project provides a simple Python-based system for analyzing student marks. The system uses a CSV dataset containing student details and marks in Maths, Python, Data Science, Statistics, and Computer.

The project performs data preprocessing, statistical analysis, visualization, and machine learning. It also provides prediction for a newly entered student and groups students according to their performance.

2. Problem Statement

Analyzing student performance manually can require significant time and effort. It can also be difficult to identify patterns and performance groups from student marks.

The aim of this project is to develop a Python-based system that can automatically load, clean, analyze, visualize, and predict student performance using data science and machine learning techniques.

3. Objectives

- The main objectives of the project are:
- To load student records from a CSV file.
- To clean and preprocess the student dataset.
- To calculate total marks and percentage.
- To assign grades and Pass/Fail results.
- To perform statistical analysis using NumPy and SciPy.
- To visualize student performance using Matplotlib and Seaborn.
- To implement Random Forest for Pass/Fail prediction.
- To predict the performance of a new student.
- To implement K-Means Clustering as a beyond-syllabus topic.
- To save the final analyzed dataset as a CSV file.

4. Technologies and Libraries Used

Technology / Library	Purpose
Python	Main programming language
Pandas	Data loading, cleaning and analysis
NumPy	Numerical calculations
SciPy	Statistical analysis
Matplotlib	Data visualization
Seaborn	Statistical graphs and heatmaps
Scikit-learn	Machine learning
Google Colab	Program development and execution
Google Drive	CSV file storage

The main machine learning algorithms used are Random Forest Classifier and K-Means Clustering.

5. System Requirements

5.1 Hardware Requirements

- Laptop or desktop computer
- Minimum 4 GB RAM
- Internet connection

5.2 Software Requirements

- Google Colab
- Python 3.x
- Google Drive
- Required Python libraries

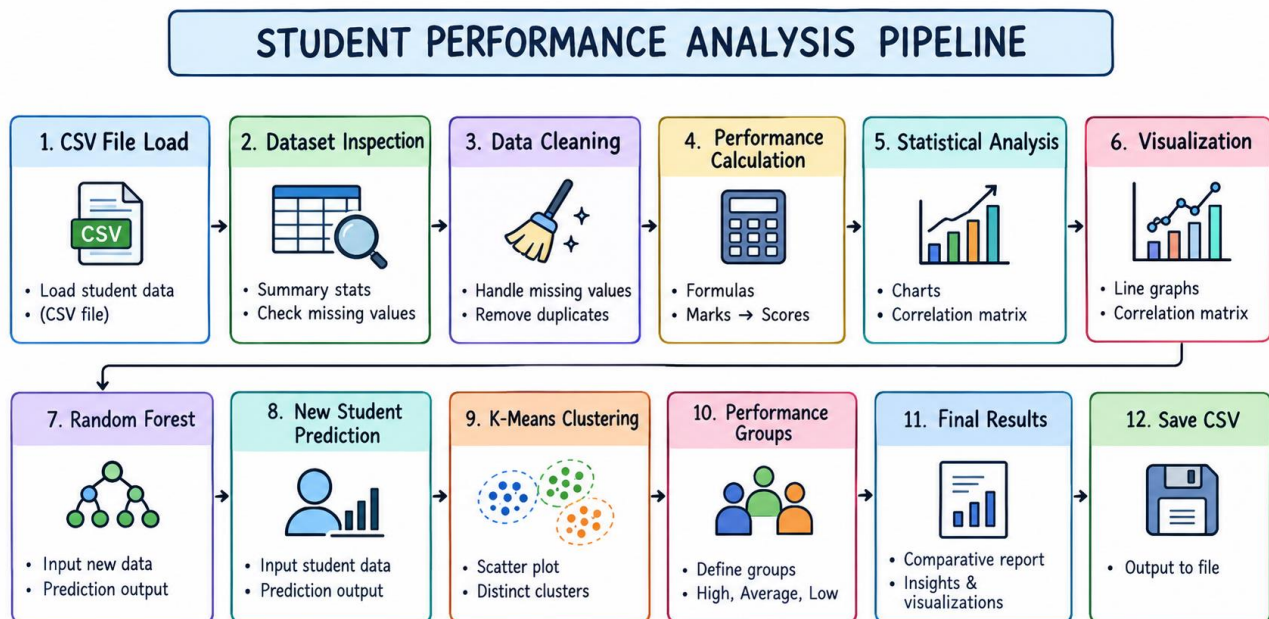
6. System Features

The major features of the system are:

- CSV dataset loading
- Dataset inspection
- Data cleaning
- Missing-value handling
- Duplicate removal
- Total and percentage calculation
- Grade calculation
- Pass/Fail classification
- Statistical analysis
- Data visualization
- Random Forest prediction
- New student prediction
- K-Means clustering
- Final CSV export

7. System Workflow

The system follows the following workflow:



First, the CSV file is loaded from Google Drive. The dataset is then inspected and cleaned. Student performance is calculated using subject marks. Statistical analysis and visualization are performed. Random Forest is used for Pass/Fail prediction. K-Means is used to group students according to their performance. Finally, the processed dataset is saved as a CSV file.

8. Dataset

The project uses a CSV dataset containing student information and subject marks. The main columns used are:

Column	Description
Roll No	Student roll number
Name	Student name
Branch	Student branch
Maths	Maths marks
Python	Python marks
Data Science	Data Science marks
Statistics	Statistics marks
Computer	Computer marks

The five subject columns are used for calculating performance and machine learning.

Code:

try:

```
df = pd.read_csv(file_path)

print("=" * 60)

print("DATASET LOADED SUCCESSFULLY")

print("=" * 60)

print("Number of Students :", len(df))

print("Number of Columns :", len(df.columns))

print("\nDataset:")

display(df)
```

except FileNotFoundError:

```
print("ERROR: File not found.")

print("Please check the file path:")

print(file_path)
```

except Exception as e:

```
print("ERROR:", e)
```

Output:

```
=====
DATASET LOADED SUCCESSFULLY
=====
Number of Students : 20
Number of Columns : 12
```

Dataset:

	Roll No	Name	Branch	Semester	Birth Date	Email	Mobile Number	Maths	Python	Data Science	Statistics
0	1	Jeel Moradiya	CE	5	8/8/2006	jeel.m@example.com	9825250062	95	85	82	
1	2	Aarav Patel	CE	5	2/14/2005	aarav.patel@example.com	9000000002	82	88	85	
2	3	Vivaan Shah	IT	5	5/21/2005	vivaan.shah@example.com	9000000003	74	81	78	
3	4	Aditya Mehta	CE	5	11/8/2004	aditya.mehta@example.com	9000000004	91	94	89	
4	5	Rohan Desai	IT	5	7/17/2005	rohan.desai@example.com	9000000005	63	69	66	
5	6	Dhruv Joshi	CE	5	1/29/2005	dhruv.joshi@example.com	9000000006	45	52	48	
6	7	Krish Trivedi	IT	5	9/12/2004	krish.trivedi@example.com	9000000007	77	73	80	
7	8	Yash Patel	CE	5	3/26/2005	yash.patel@example.com	9000000008	56	62	59	
8	9	Harsh Shah	IT	5	12/3/2004	harsh.shah@example.com	9000000009	88	84	91	
9	10	Dev Mehta	CE	5	6/19/2005	dev.mehta@example.com	9000000010	39	46	42	
10	11	Parth Desai	IT	5	10/27/2004	parth.desai@example.com	9000000011	69	71	67	

9. Data Analysis

Code:

```
df["Total"] = df[subjects].sum(axis=1)
df["Percentage"] = df["Total"] / len(subjects)
def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B"
    elif percentage >= 60:
        return "C"
    elif percentage >= 50:
        return "D"
    elif percentage >= 40:
        return "E"
    else:
        return "F"
def calculate_result(percentage):
    return "PASS" if percentage >= 40 else "FAIL"
df["Grade"] = df["Percentage"].apply(calculate_grade)
df["Result"] = df["Percentage"].apply(calculate_result)
display(df[["Roll No", "Name", "Total", "Percentage", "Grade", "Result"]])
```

Output:

	Roll No	Name	Total	Percentage	Grade	Result
0	1	Jeel Moradiya	434	86.8	A	PASS
1	2	Aarav Patel	425	85.0	A	PASS
2	3	Vivaan Shah	389	77.8	B	PASS
3	4	Aditya Mehta	461	92.2	A+	PASS
4	5	Rohan Desai	331	66.2	C	PASS
5	6	Dhruv Joshi	243	48.6	E	PASS
6	7	Krish Trivedi	383	76.6	B	PASS
7	8	Yash Patel	296	59.2	D	PASS
8	9	Harsh Shah	438	87.6	A	PASS
9	10	Dev Mehta	214	42.8	E	PASS
10	11	Parth Desai	350	70.0	B	PASS
11	12	Manav Joshi	468	93.6	A+	PASS
12	13	Meet Patel	271	54.2	D	PASS
13	14	Jay Shah	379	75.8	B	PASS
14	15	Shiv Mehta	426	85.2	A	PASS

10. Statistical Analysis

Statistical analysis is performed using NumPy and SciPy. The system calculates:

- Average marks
- Highest mark
- Lowest mark
- Mean percentage
- Median percentage
- Standard deviation
- Mode percentage

Code:

```
marks = df[subjects].to_numpy()
print("Average Marks:", np.mean(marks))
print("Highest Mark:", np.max(marks))
print("Lowest Mark:", np.min(marks))
print("\nStatistics:")
print("Mean Percentage:", df["Percentage"].mean())
print("Median Percentage:", df["Percentage"].median())
print("Standard Deviation:", df["Percentage"].std())
# SciPy
mode_result = stats.mode(df["Percentage"], keepdims=True)
print("Mode Percentage:", mode_result.mode[0])
```

Output:

```
Average Marks: 72.13
Highest Mark: 96
Lowest Mark: 38

Statistics:
Mean Percentage: 72.13000000000001
Median Percentage: 76.19999999999999
Standard Deviation: 16.54967323576288
Mode Percentage: 42.8
```

11. Data Visualization

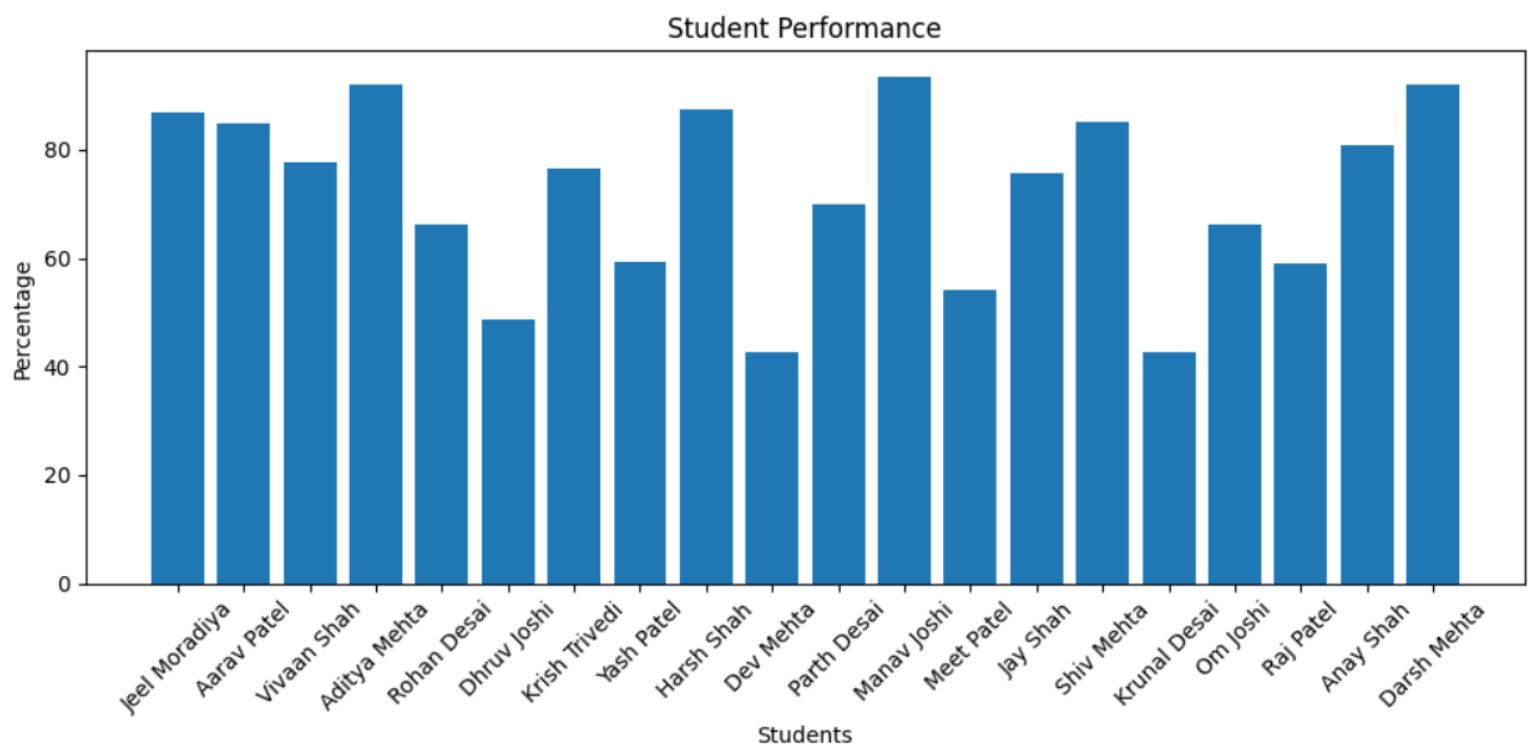
Data visualization is used to represent student performance graphically. The project uses Matplotlib and Seaborn to create the following visualizations:

11.1 Student Performance Bar Chart

Code:

```
plt.figure(figsize=(10, 5))
plt.bar(df["Name"], df["Percentage"])
plt.xlabel("Students")
plt.ylabel("Percentage")
plt.title("Student Performance")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output:

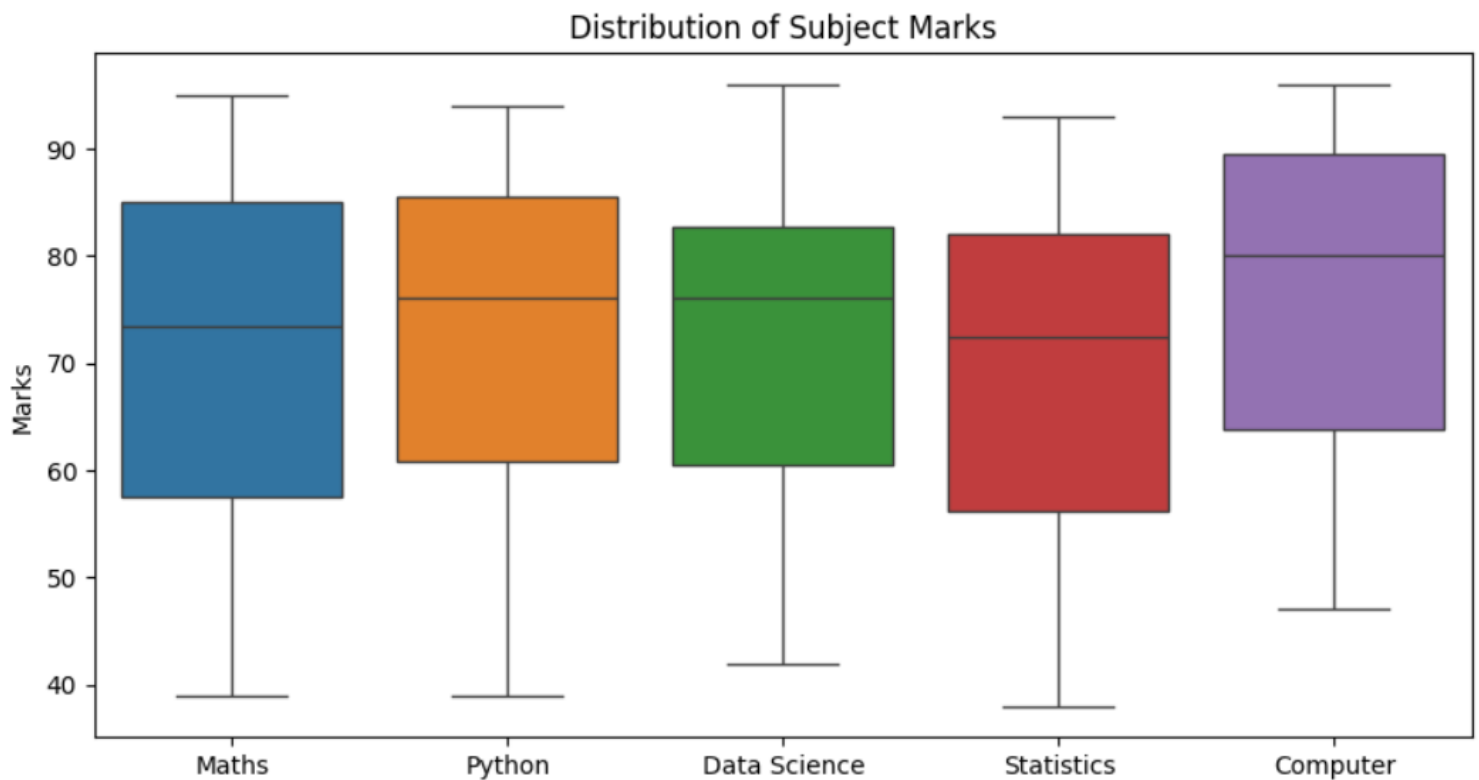


11.2 Subject Marks Box Plot

Code:

```
plt.figure(figsize=(10, 5))
sns.boxplot(data=df[subjects])
plt.title("Distribution of Subject Marks")
plt.ylabel("Marks")
plt.show()
```

Output:



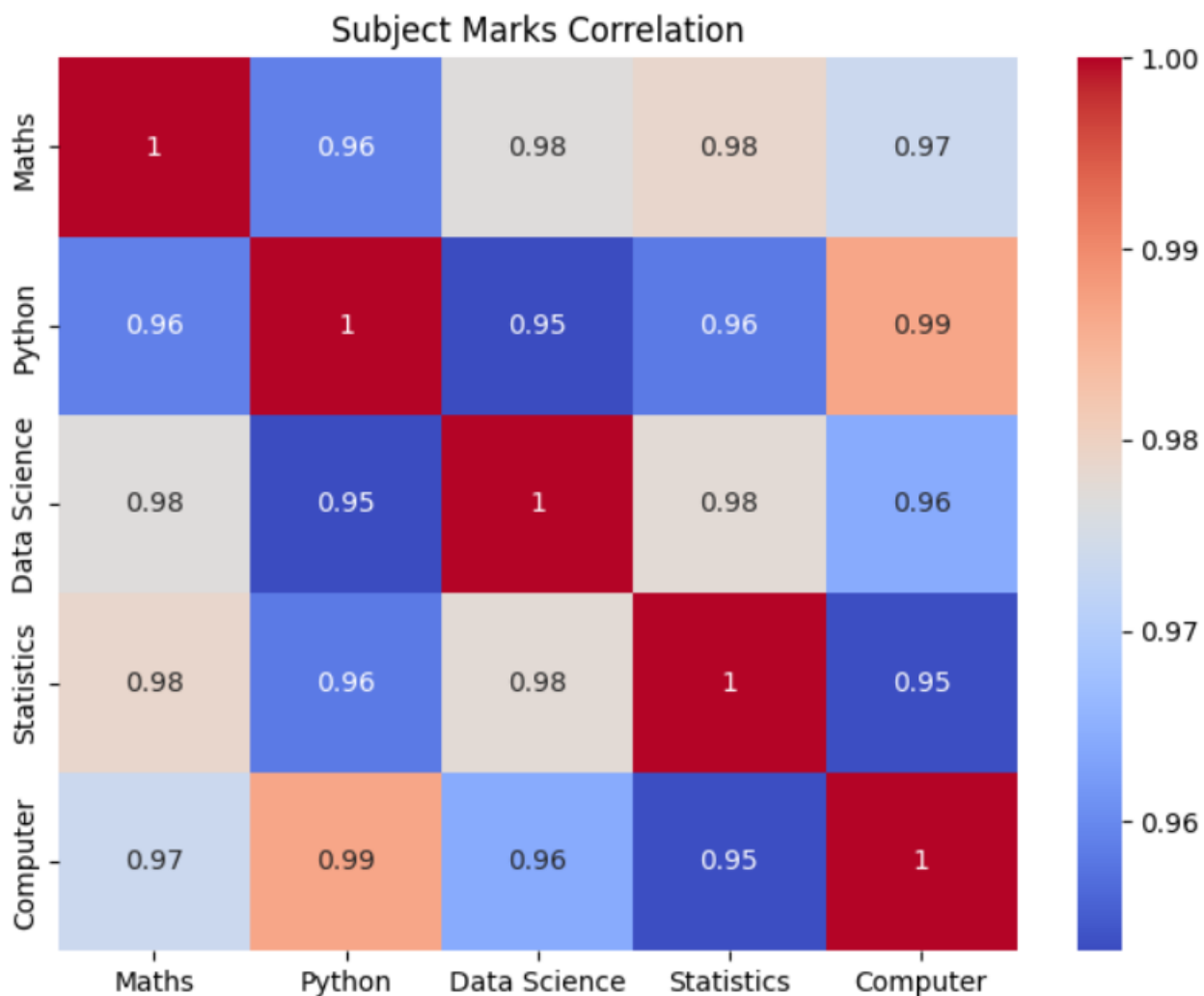
11.3 Correlation Heatmap

The heatmap shows the correlation between the five subjects.

Code:

```
plt.figure(figsize=(8, 6))
sns.heatmap(
    df[subjects].corr(),
    annot=True,
    cmap="coolwarm"
)
plt.title("Subject Marks Correlation")
plt.show()
```

Output:



12. Machine Learning

The project uses Random Forest Classifier for student Pass/Fail prediction. The five subject marks are used as input features:

- Maths
- Python
- Data Science
- Statistics
- Computer

Code:

```
X = df[subjects]
y = df["Result"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Accuracy:", round(accuracy * 100, 2), "%")
```

Output:

```
Random Forest Accuracy: 100.0 %
```

13. New Student Prediction

The system provides a simple method for predicting the result of a new student. The user enters:

- Roll No
- Student Name
- Mobile Number
- Birth Date
- Email
- Maths marks
- Python marks
- Data Science marks
- Statistics marks
- Computer marks

Output:

```
===== NEW STUDENT DETAILS =====
Enter Roll No: 21
Enter Student Name: Krutik
Enter Mobile Number: 9863125478
Enter Birth Date (DD-MM-YYYY): 05-05-2006
Enter Email: kru08@example.com

Enter Subject Marks
Enter Maths marks: 95
Enter Python marks: 96
Enter Data Science marks: 85
Enter Statistics marks: 80
Enter Computer marks: 85

===== STUDENT DETAILS =====
Roll No: 21
Name: Krutik
Mobile: 9863125478
Birth Date: 05-05-2006
Email: kru08@example.com

===== PERFORMANCE =====
Total Marks: 441.0
Percentage: 88.2
Grade: A
Predicted Result: PASS
```

14. Beyond Syllabus Topic: K-Means Clustering

Code:

```
cluster_data = df[subjects]

scaler = StandardScaler()

scaled_data = scaler.fit_transform(cluster_data)

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(scaled_data)

print("Student Clusters:")
display(df[["Name", "Percentage", "Cluster"]])

cluster_average = df.groupby("Cluster")["Percentage"].mean()

print("Average Percentage of Each Cluster:")
print(cluster_average)

sorted_clusters = cluster_average.sort_values().index

cluster_names = {
    sorted_clusters[0]: "Low Performance",
    sorted_clusters[1]: "Average Performance",
    sorted_clusters[2]: "High Performance"
}
```

```
df["Performance_Group"] = df["Cluster"].map(cluster_names)
```

```
display(
```

```
df[["Name", "Percentage", "Cluster", "Performance_Group"]])
```

Output:

Average Percentage of Each Cluster:

Cluster

0 88.914286

1 51.100000

2 73.371429

Name: Percentage, dtype: float64

	Name	Percentage	Cluster	Performance_Group
0	Jeel Moradiya	86.8	0	High Performance
1	Aarav Patel	85.0	0	High Performance
2	Vivaan Shah	77.8	2	Average Performance
3	Aditya Mehta	92.2	0	High Performance
4	Rohan Desai	66.2	2	Average Performance
5	Dhruv Joshi	48.6	1	Low Performance
6	Krish Trivedi	76.6	2	Average Performance
7	Yash Patel	59.2	1	Low Performance
8	Harsh Shah	87.6	0	High Performance
9	Dev Mehta	42.8	1	Low Performance
10	Parth Desai	70.0	2	Average Performance
11	Manav Joshi	93.6	0	High Performance

Code:

```
plt.figure(figsize=(8, 6))
```

```
sns.scatterplot(
```

```
data=df,
```

```
x="Python",
```

```
y="Data Science",
```

```
hue="Performance_Group",
```

```
s=100
```

```
)
```

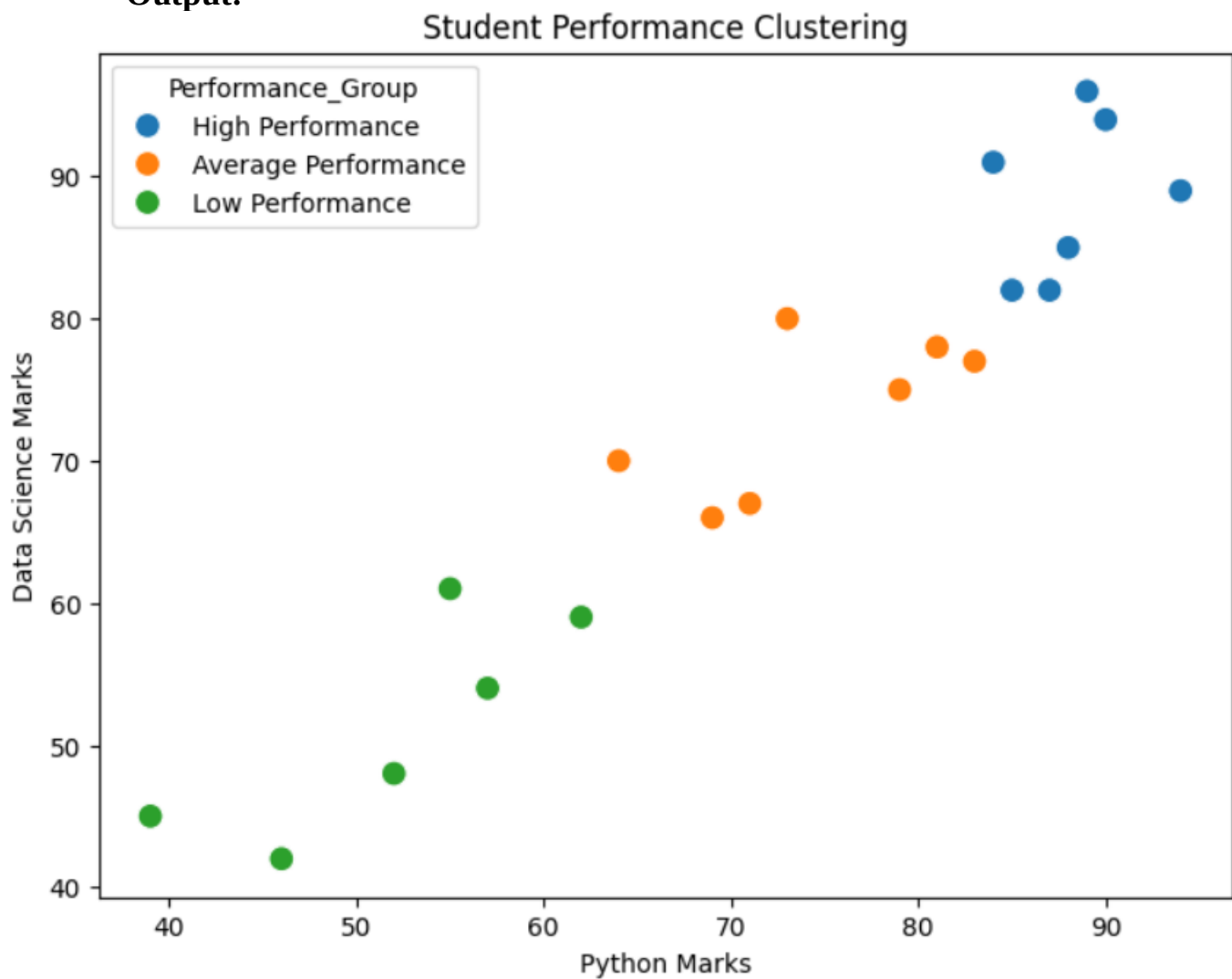
```
plt.title("Student Performance Clustering")
```

```
plt.xlabel("Python Marks")
```

```
plt.ylabel("Data Science Marks")
```

```
plt.show()
```

Output:



15. Implementation

The project is implemented using Python in Google Colab. The implementation consists of the following steps:

1. Mount Google Drive.
2. Specify the CSV file path.
3. Import required libraries.
4. Load the dataset using Pandas.
5. Display dataset information.
6. Define the subject columns.
7. Clean and preprocess the data.
8. Calculate total marks and percentage.
9. Assign grades and Pass/Fail results.
10. Perform statistical analysis.
11. Generate visualizations.
12. Train the Random Forest model.
13. Predict the result of a new student.
14. Apply K-Means Clustering.
15. Create performance groups.
16. Display final results.
17. Save the processed dataset as a CSV file.

16. Important Code

16.1 Calculate Total and Percentage

16.2 Random Forest

16.3 K-Means Clustering

17. Results and Observations

The Student Performance Analytics system successfully performs data analysis and machine learning on the student dataset.

The system successfully cleans the dataset and calculates total marks, percentage, grade, and Pass/Fail results. Statistical analysis provides information about the overall performance of the students.

The generated graphs help in understanding subject-wise and student-wise performance. Random Forest is used to predict Pass/Fail results, while K-Means groups students according to their academic performance.

18. Advantages

The main advantages of the project are:

- Simple and easy to use.
- Reduces manual calculations.
- Handles missing data.
- Provides statistical analysis.
- Provides graphical visualization.
- Uses machine learning for prediction.
- Allows new student prediction.
- Groups students according to performance.
- Saves the final dataset for future use.

19. Limitations

The project has the following limitations:

- Prediction accuracy depends on the available dataset.
- A small dataset may produce unstable machine learning results.
- The CSV file must follow the required format.
- Only predefined subjects are currently supported.
- The current system does not have a graphical user interface.
- The Random Forest model uses subject marks as its main prediction features.

20. Future Scope

The project can be further improved by adding a graphical user interface or web-based dashboard.

Additional information such as attendance, assignments, internal marks, and semester-wise performance can be included.

Database integration can be added to store student records. More machine learning algorithms can also be tested and compared.

The system can also be extended to automatically generate individual student performance reports.

21. Conclusion

The Student Performance Analytics project demonstrates the practical application of Python for data analysis, statistics, visualization, and machine learning.

The system successfully loads and cleans student data, calculates academic performance, performs statistical analysis, and generates different visualizations.

Random Forest Classifier is used to predict Pass/Fail results, while K-Means Clustering is implemented as the beyond-syllabus topic to group students into different performance categories.

The project provides a simple approach to analyzing student performance and demonstrates how Python and machine learning can be applied to an educational dataset.

References

- Python Documentation
- Pandas Documentation
- NumPy Documentation
- SciPy Documentation
- Matplotlib Documentation
- Seaborn Documentation
- Scikit-learn Documentation
- Google Colab –

<https://colab.research.google.com/drive/1FWcnZsrW9DPnL59NW6tDJDZzVp4wbNXj?usp=sharing>